



CIRCUITOS DIGITALES Y MICROCONTROLADORES 2026

Facultad de Ingeniería
UNLP

Máquinas de Estados Finitos
y su implementación en Software

Ing. José Juárez

Implementación: MEF en software

- Del análisis de la especificación podemos extraer los siguientes elementos a implementar:

1) Definir el conjunto de estados

estados: (S0, S1,...,Sn)

2) Definir el conjunto de entradas (alfabeto)

entradas: (i0, i1,...,im)

3) Definir el conjunto de salidas

salidas: (O0, O1,...,Ok)

4) Definir una función de transición de estados

tabla : [1...n] [1...m]

5) Definir procedimiento para establecer el estado inicial

```
Iniciar_MEF {  
    estado=Si    //i e 0..n  
    salida=Oj    //j e 0..k  
}
```

Implementación: MEF en software

6) Definir procedimiento para actualizar la MEF

```
Actualizar_MEF() {  
    Leer(&entradas);  
    estado=tabla[estado][entradas];  
    Act_Salidas(estados,entradas);  
}
```

Implementación
por tablas



7) Definir procedimientos para ejecutar la MEF MEF *(es independiente del modelo y depende de la arquitectura del software: time-triggered o event – triggered)*

Sin temporizar o
periódica cada T



```
Ejecutar_MEF_x{  
    Iniciar_MEF_x();  
    repetir siempre o cada T [seg]{  
        Actualizar_MEF_x();  
    }  
}
```

Implementación: switch-case

```
void ActualizarMEF(void)
{
    X=leerEntradas();
    switch(estado) {
        Case S0:
            if(X==1)
                { estado=S1; Z=1;}
            else
                { estado=S0; Z=0;}
            break;
        Case S1:
            if(X==1)
                { estado=S0; Z=0;}
            else
                { estado=S1; Z=1;}
            break;
    }
}
```

Si hay múltiples
entradas => se
puede poner otro
switch-case en
lugar de if-else



Implementación: switch-case

*Definición y
Declaración de
Variables de estado*



```
typedef enum{S0,S1}state;  
state estado;
```

Método de Inicialización →

```
Iniciar_MEF() {  
    estado=S0; Z=0;  
}
```

Ejecución de la MEF →



```
void main(void) {  
    Iniciar_MEF();  
  
    while(1) {  
        //se puede repetir cada T  
        //delay(T);  
        Actualizar_MEF();  
    }  
}
```

Implementación: con punteros a función

```
typedef enum{S0,S1} state;

void fS0(void);
void fS1(void);
void (*MEF[]) (void)={fS0,fS1};
state estado;
```

```
void main(void) {
    Iniciar_MEF();
    while(1) {
        Actualizar_MEF();
    }
}
```

```
void ActualizarMEF(void) {
    X=leerEntradas();
    (*MEF[estado])();
}
```

```
void fS0(void)
{
    if(X==1) {
        estado=S1; Z=1;}
    else {
        estado=S0; Z=0;}
}
```

```
void fS1(void)
{
    if(X==1) {
        estado=S0; Z=0;}
    else {
        estado=S1; Z=1;}
}
```

//Ejecuta la función correspondiente

Implementación: MEF en Software

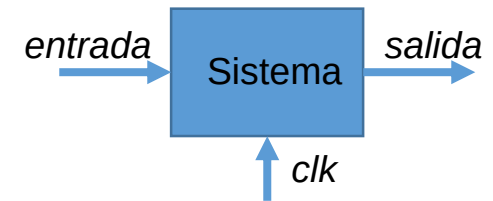
- **MEF Temporizadas:** Actualización periódica con Timer (polling periódico)

```
typedef enum={S0,S1}state;  
state estado;
```

```
void main(void) {  
    Iniciar_MEF();
```

```
    while(1) {  
        if (FLAG_TIMER) {  
            Actualizar_MEF();  
            FLAG_TIMER=0;  
        }  
        sleep();  
    }
```

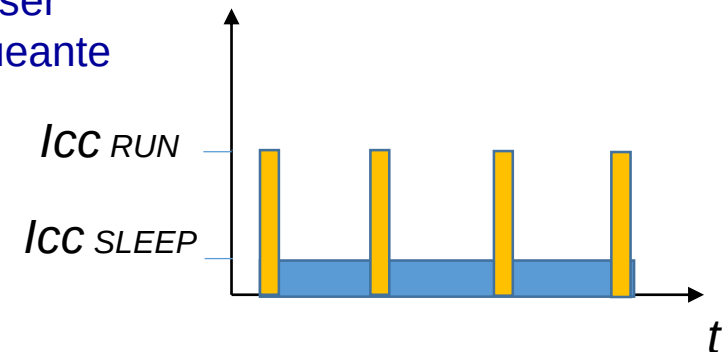
Similar a la implementación en Hardware



Flag modificado x ISR de Timer

Debe ser
No-bloqueante

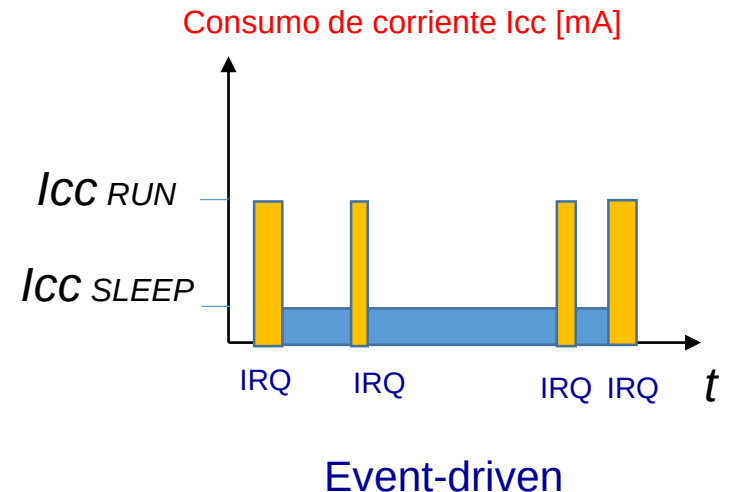
Consumo de corriente I_{CC} [mA]



Implementación: MEF en Software

- **MEF controladas por eventos:** Actualización por eventos no periódicos generalmente (puede derivar en una arquitectura con múltiples interrupciones)

```
Ejecutar_MEF{  
  
    Iniciar_MEF();  
    repetir siempre{  
        sleep();  
        Actualizar_MEF();  
    }  
}  
  
ISR(IRQ_request) {  
    Leerentradas();  
}
```



Implementación: MEF en Software

- **MEF jerárquicas:** Un estado puede contener otra MEF

```
void ActualizarMEF(void)
{
    switch (estado) {
        Case S0:
            Actualizar_MEF_S0(&params);
            break;
        Case S1:
            if (X==1)
                { estado=S0; Z=0; }
            else
                { estado=S1; Z=1; }
            break;
    }
}
```

Tiene que heredar
todas las variables
del estado
superior

Implementación: MEF en Software

- **MEF cooperativas** varias MEF independientes en ejecución “simultánea”

```
void main(void) {  
    Iniciar_MEF_1();  
    Iniciar_MEF_2();  
  
    while(1) {  
        Actualizar_MEF_1();  
        Actualizar_MEF_2();  
    }  
}
```

- 1-No-bloqueantes
- 2-Independientes (2 diagramas)
- 3-Se comunican x mensajes asincrónicos
- 4-Se ejecutan hasta completar (run to completion)
- 5-Son cooperativas (corrutinas)

- **Statecharts:** Método gráfico (especificación UML) para especificar MEFs generalizadas (jerárquicas y concurrentes)

Ejemplo 1

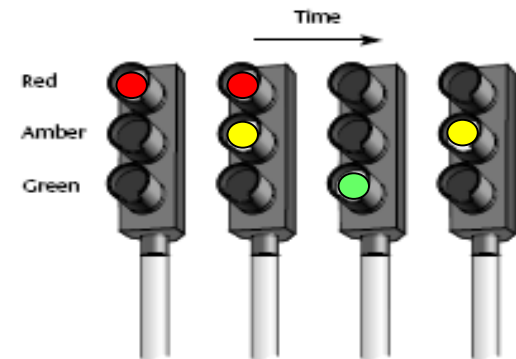
- Semáforo (MEF temporizada)

Estados y salidas (Moore):

- 1-rojo
- 2-rojo-amarillo
- 3-verde
- 4-amarillo

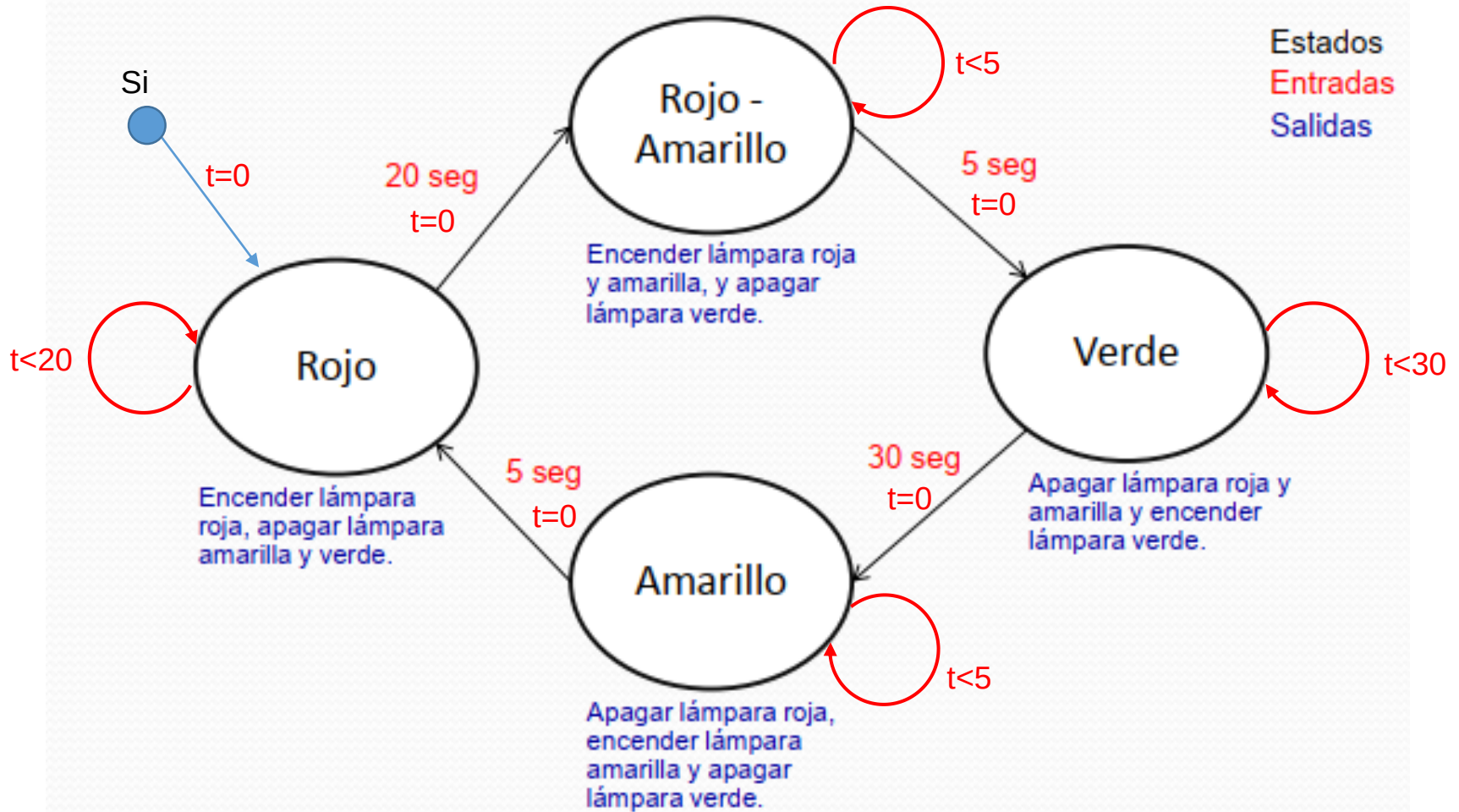
Cambio de estados x evento tiempo (entrada)

- Rojo: 20seg
- Rojo-amarillo: 5seg
- verde: 30seg
- Amarillo: 5 seg



Semáforo: Implementación MEF

Diagrama de estados del ejemplo de semáforo:



Semáforo: Implementación MEF

```
// Posibles estados del sistema
typedef enum {RED, RED_AMBER, GREEN, AMBER} eLight_State;

// Tiempo de duración de cada estado
// (tiempo en segundos)
//
#define RED_DURATION 20
#define RED_AMBER_DURATION 5
#define GREEN_DURATION 30
#define AMBER_DURATION 5
```

Semáforo: Implementación MEF

```
// ----- variables privadas-----

static eLight_State Light_state; // Estado actual del sistema
static long Time_in_state; // Tiempo en el estado

/*-----*-
TRAFFIC_LIGHTS_Init()
-*-----*/

void TRAFFIC_LIGHTS_Init ( eLight_State START_STATE)
{
    Light_state = START_STATE; // poner un estado inicial
}

void main(void)
{
    MCU_Init();

    TRAFFIC_LIGHTS_Init (RED); // inicializar MEF

    while(1){
        TRAFFIC_LIGHTS_Update(); // MEF temporizada aprox. c/1 seg
        delay_ms(1000);
    }
}
```

Semáforo Implementación MEF

```
void TRAFFIC_LIGHTS_Update(void)
```

```
{
```

```
switch (Light_state)
```

```
{
```

```
case RED:
```

```
Red_light = ON;
```

← *salidas*

```
Amber_light = OFF;
```

```
Green_light = OFF;
```

← *entrada*

```
if(++Time_in_state == RED_DURATION)
```

```
{
```

```
Light_state = RED_AMBER;
```

← *Cambio estado*

```
Time_in_state = 0;
```

```
}
```

```
break;
```

```
case RED_AMBER:
```

```
Red_light = ON;
```

```
Amber_light = ON;
```

```
Green_light = OFF;
```

```
if(++Time_in_state == RED_AMBER_DURATION)
```

```
{
```

```
Light_state = GREEN;
```

```
Time_in_state = 0;
```

```
}
```

```
break;
```

(Moore)

```
case GREEN:
```

```
Red_light = OFF;
```

```
Amber_light = OFF;
```

```
Green_light = ON;
```

```
if(++Time_in_state == GREEN_DURATION)
```

```
{
```

```
Light_state = AMBER;
```

```
Time_in_state = 0;
```

```
}
```

```
break;
```

```
case AMBER:
```

```
Red_light = OFF;
```

```
Amber_light = ON;
```

```
Green_light = OFF;
```

```
if(++Time_in_state == AMBER_DURATION)
```

```
{
```

```
Light_state = RED;
```

```
Time_in_state = 0;
```

```
}
```

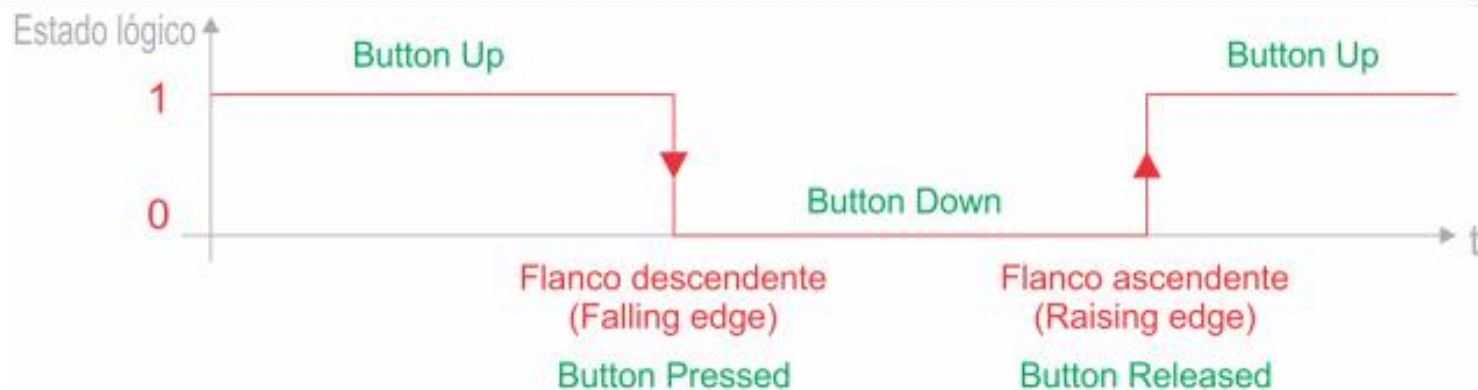
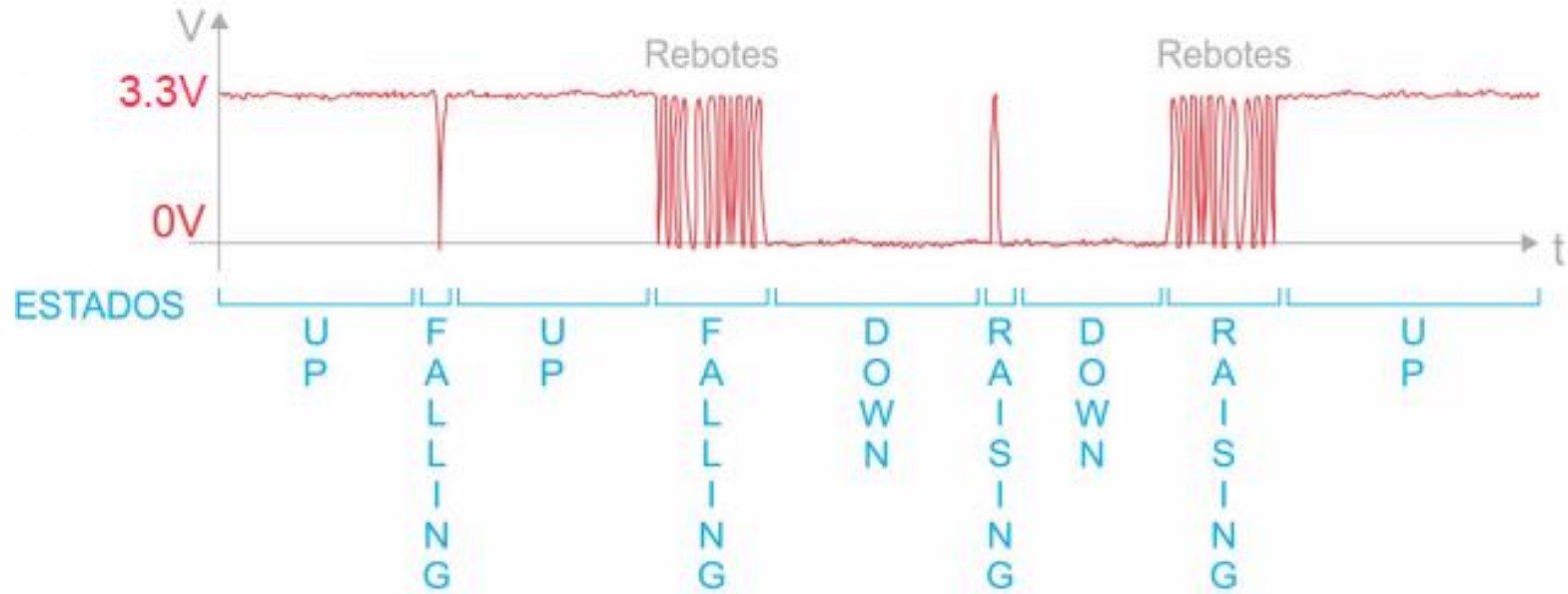
```
break;
```

```
}
```

```
}
```

Ejemplo 2

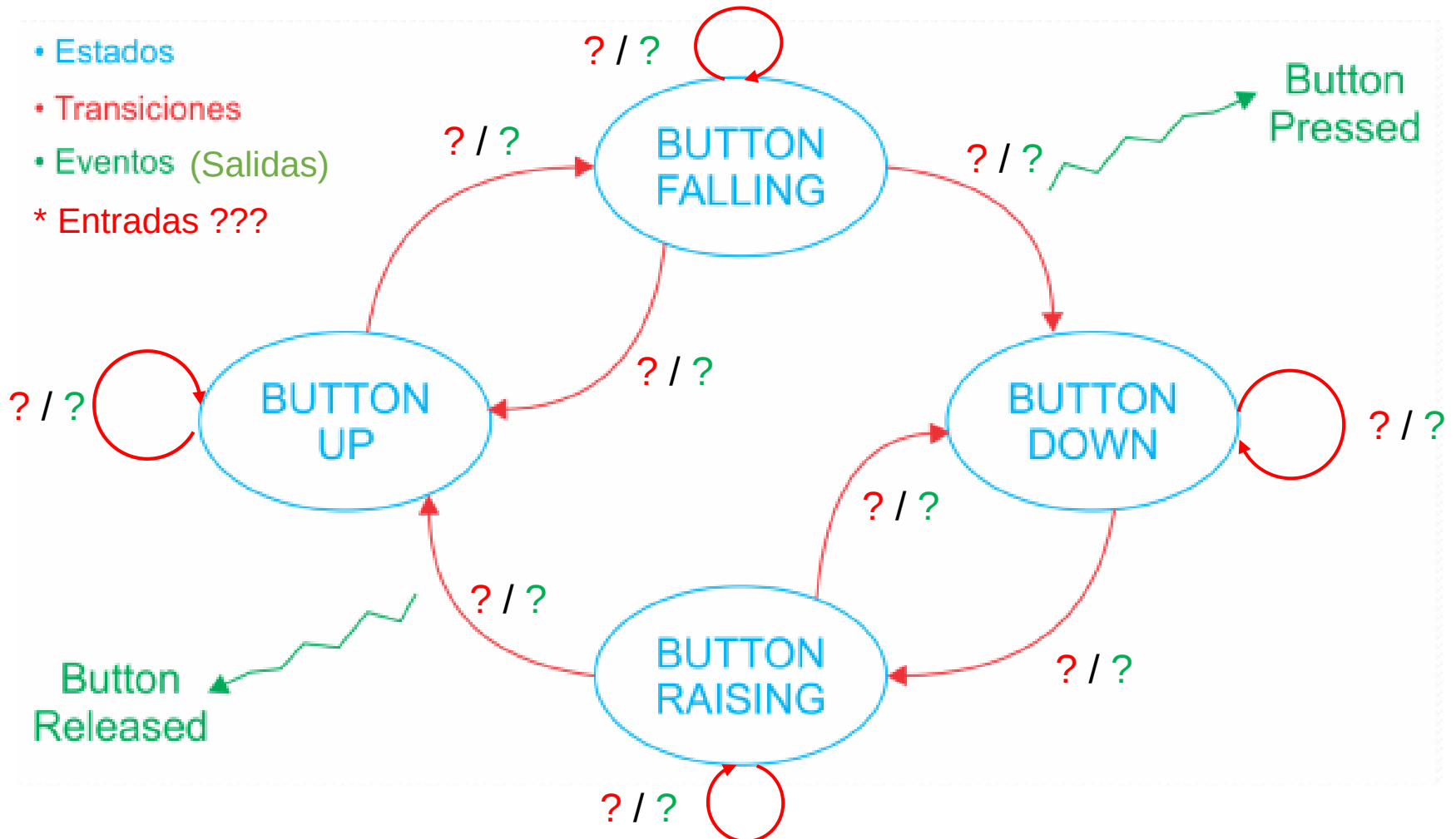
- Detección de pulsador con anti-rebote



Ejemplo 2

- Implementación MEF (Mealy)

Tarea: Completar especificación



Ejemplo 2: Detección de pulsador con anti-rebote

- En el contexto de una aplicación MEF que requiere como entrada pulsadores:

```
Actualizar_MEF_1() {  
    Leer(&entrada);  
    estado=tabla[estado][entrada];  
    Act_Salidas(estado, entrada);  
}
```

- El algoritmo anti-rebote sería una sub_MEF :

```
Actualizar_MEF_1() {  
    Actualizar_MEF_2(&entrada);  
    estado=tabla[estado][entrada];  
    Act_Salidas(estado, entrada);  
}
```

Devuelve Press o
Released

